SafeCurves / SECG exotic-trapdoor slots (2026-10-17)

Status. Protocol with two executed rounds and registered follow-ups. No deployed-curve weakness beyond the census rows is claimed; no key recovery. Continues `research/nist_trapdoor_slot_program_20261011/` by asking which trapdoor families could **pass** the SafeCurves (or weaker SECG) criteria.

Question

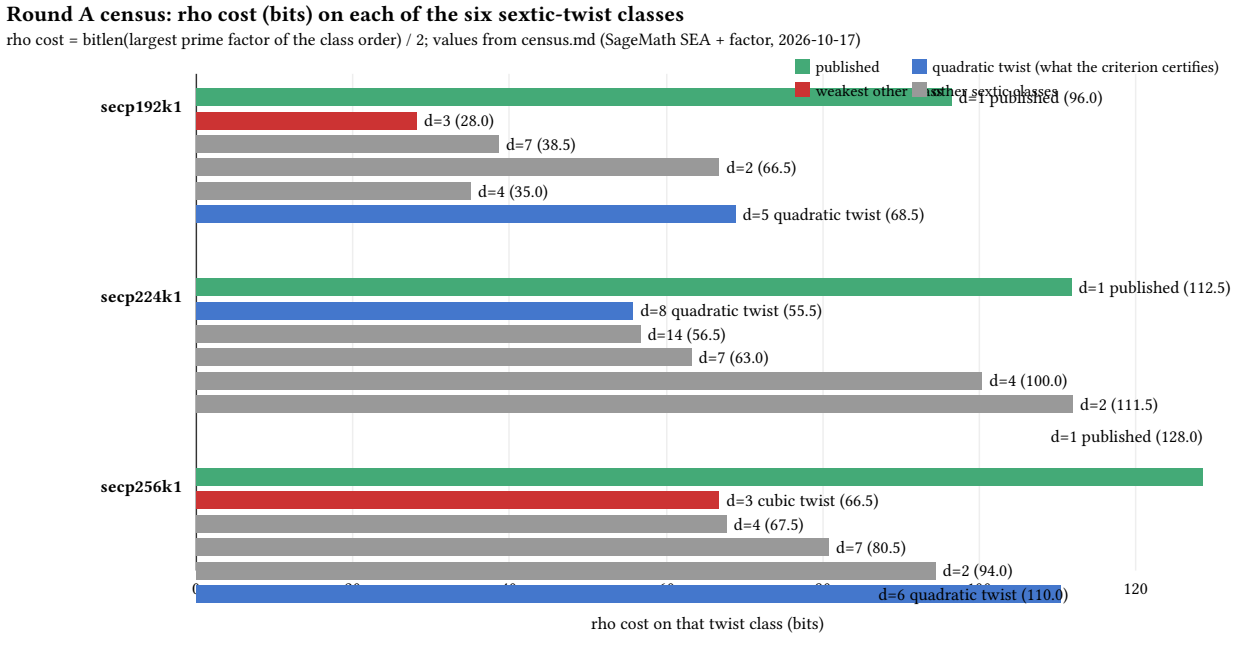
SafeCurves certifies, per curve: ECDLP rho cost, no pairings transfer, CM discriminant `|D| > 2^100`, twist security (rho on the quadratic twist), ladder support, completeness, rigidity of generation. SECG requires far less, with no twist-security or rigidity requirement at all. If a designer wanted a curve that passes these checks while retaining a secret intermediate-cost or sub-exponential advantage, which criterion cannot see it?

Criteria-versus-trapdoor map (slots T1-T6)

slot	trapdoor family	killed by	can it pass SafeCurves?
T1	group representation up to isogeny (hidden homomorphism / model-dependent solver advantage, Teske class)	nothing in SafeCurves; discriminant kills class-group maps, transfer kills pairings	**yes** - the only slot with no public check
T2	special prime / non-pairing transfer	transfer criterion (pairings only)	yes
T3	grind onto a secret coefficient-recognisable class (BADA55 pattern)	rigidity in spirit only; documented generation is not ungrindable generation	yes
T4	quadratic-twist weakness	SafeCurves twist criterion	no (SafeCurves); **yes for SECG**
T5	**sextic/quartic twist weakness on `j = 0` / `j = 1728` models**	nothing: the twist criterion certifies one of six classes	**yes** - this round's finding
T6	base-point protocol trapdoor (Dual_EC class)	not an ECDLP criterion anywhere	yes, not curve-mathematical

Round A (executed): sextic-twist-orbit census of the SECG `j = 0` curves

Command: `sage twist_orbit_census.py` (SEA point counting; BPSW probable primes; `census.md` / `census.json` verbatim output). For `p = 1 mod 6`, the curves `y^2 = x^3 + b*d` fall into six `F_p`-isomorphism classes with six **different** orders `{p+1 +/- t_i}`, the traces satisfying the classical `j = 0` CM relation `t_1 = t_2 + t_3` (verified on secp256k1 from the census data; the whole orbit is derivable from the public `(p, n)` alone).



Rho cost on each of the six sextic-twist classes, from `census.md`. Green: published class; blue: the quadratic twist the criterion certifies; red: weakest other class.

Measured summary (rho cost = half the bit length of the largest prime factor of the class order):

curve	published	quadratic twist (certified)	weakest other sextic class
secp192k1	2^96	2^68.5	**2^28.0** (witness d = 3)
secp224k1	2^112.5	2^111.5	**2^55.5** (witness d = 8; d = 14 at 2^56.5 is another)
secp256k1	2^128	2^110	**2^65.5** (witness d = 3, a cubic twist)

Reading notes, kept honest: the quadratic twist of secp256k1 is **strong** (2^110); the well-known weak secp256k1 twist figure belongs to a **cubic** twist, which a SafeCurves-style check never looks at. x-only ladders reach only the quadratic twist, so Montgomery-style implementations are covered; implementations that accept a supplied `y` without validation can be fed points on any class (the classic invalid-curve setting). The marginal content of T5 is that for `j = 0` curves the weak class is a published-parameter-determined one at a measured cost far below the certified twist. This is an implementation-level observation about what "twist security" certifies, reported as such.

Round B (executed): the grind is nearly free (toy scale)

Command: `sage sextic_grind_demo.py`. The designer picks `p = 1 mod 6` (free under SECG-style generation) and the sextic class of `b`. Scaled SafeCurves-style acceptance: published class and its quadratic twist both have largest prime factor at least `0.79 * bitlen(p)`; the withheld trapdoor is another class at most `0.40 * bitlen(p)`.

size	primes scanned	published	twist class	weakest sibling (rho)
48 bits	2	48 bits	40 bits	13 bits (2^6.5)
96 bits	3	95 bits	79 bits	28 bits (2^14)

Registered prediction: at 256 bits the expected number of `p = 1 mod 6` primes to scan before such a pair is below `2^12`; falsified if none exists in `2^13` scanned primes. Round C (pending, native Rust per the repository's no-Python rule) runs this and cross-checks the census via the CM trace relation instead of SEA.

What survives for SafeCurves-conforming models

Curves that pass the ladder criterion are Montgomery/Edwards models with `j != 0, 1728`; those have exactly one nontrivial twist, which the twist criterion fully covers. So T5 lives only on short-Weierstrass `j = 0` / `j = 1728` models - exactly the SECG `secp*k1` family - and the exotic search for SafeCurves-grade criteria reduces to T1 (representation) and T3 (rigidity that is not manipulation-proof). This sharpens the NIST program's S1 conclusion: the only slot with no public check at either criteria level is the representation itself.

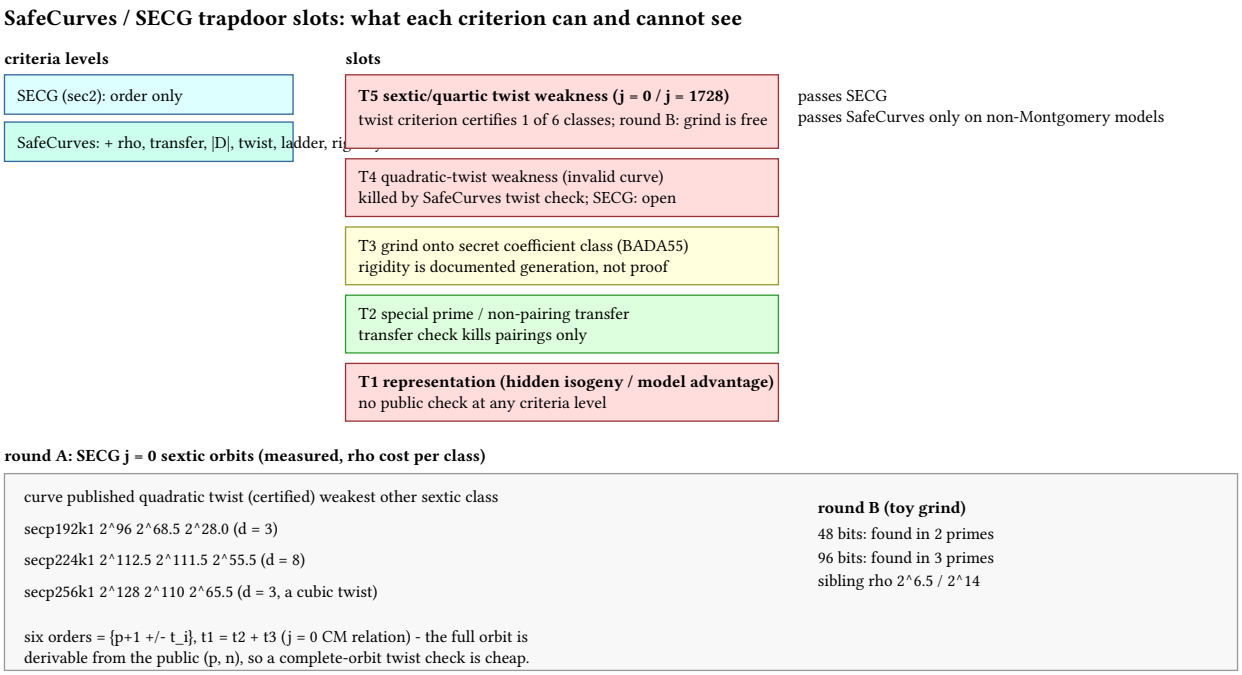


Figure 1: Criteria levels vs slots, with round-A/B numbers.

Registered follow-ups

- **R1 (round C, native).** Rewrite rounds A and B in Rust (SEA or the CM trace derivation `4p = L^2 + 27M^2`, repository factoring tools); run the 256-bit grind prediction; register the full twist-orbit check as a public rigidity-adjacent criterion candidate.
- **R2.** Quartic analogue at `j = 1728` (`y^2 = x^3 + b`, `p = 1 mod 4`): census and grind on the four-class orbit; criteria-completion only.
- **R3.** Reachability experiment: instrumented x-only ladder and a y-accepting implementation on the round-B 96-bit curve, measuring which classes each can be pushed onto.

Claim hygiene

Synthetic fixtures only; no deployed-curve-security claim beyond the census rows; no key recovery; the grind demonstration is a toy-scale construction, not evidence that any SECG curve was so generated. The twist-orbit facts are measured; the T1/T3 conclusions are derived reasoning over published criteria; the 256-bit grind prediction is a registered extrapolation until round C lands.

References

- D. J. Bernstein, T. Lange, **SafeCurves** (safecurves.cr.yp.to), criteria tables.
- SEC 2 v2.0, **Recommended Elliptic Curve Domain Parameters**.
- E. Teske, **An elliptic curve trapdoor system**, J. Cryptology 19(1), 2006.
- D. J. Bernstein et al., **How to manipulate curve standards**, SSR 2015 (BADA55).
- Repository: `research/nist_trapdoor_slot_program_20261011/PROTOCOL.md`, `RESEARCH_P256_BACKDOOR_SPECULATION.md`.
- J. H. Silverman, **The Arithmetic of Elliptic Curves** (twists and $\mathbb{F}_p$-isomorphism classes).